

ĐẠI HỌC QUỐC GIA TP HCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



Thực hành HDFS

Bài thực hành 1

Nhập môn Dữ liệu lớn

Sinh viên thực hiện:

Văn Thị Diễm My - 23127091

Giáo viên hướng dẫn:

TS. Nguyễn Ngọc Thảo

ThS. Huỳnh Lâm Hải Đăng

CN. Trần Huy Bân

Tháng 07 năm 2026

Mục lục

1	Giới thiệu	1
2	Cài đặt môi trường	2
2.1	Cài đặt Java	2
2.2	Cấu hình SSH	2
2.3	Cài đặt Hadoop	3
3	Cấu hình các tập tin Hadoop	3
3.1	hadoop-env.sh	3
3.2	core-site.xml	3
3.3	hdfs-site.xml	4
3.4	mapred-site.xml	5
3.5	yarn-site.xml	5
4	Khởi tạo và khởi động cụm Hadoop	6
4.1	Format NameNode	6
4.2	Khởi động các dịch vụ Hadoop	6
4.3	Kiểm tra Web UI	7
5	Thao tác trên HDFS	9
5.1	Tạo thư mục trên HDFS và người dùng	9
5.2	Tải dữ liệu lên HDFS	9
5.3	Kiểm tra kết quả	10
6	Chạy chương trình Verification	10
6.1	Lệnh thực hiện	10
6.2	Kết quả thực hiện	11
6.3	Kết quả	11
7	Chương trình WordCount sử dụng MapReduce	11
7.1	Yêu cầu bài toán	11
7.2	Thiết kế chương trình	12
7.2.1	Tập tin build.sbt	12
7.2.2	Lớp FilteredMapper	13
7.2.3	Lớp IntSumCombiner	14
7.2.4	Lớp IntSumReducer	15
7.2.5	WordCount	16
7.3	Thực thi chương trình	16

7.4	Kết quả trên HDFS	18
8	Lỗi gặp phải và cách khắc phục	19
8.1	Lỗi JAVA_HOME không tồn tại	19
8.2	Lỗi Incompatible ClusterID	20
9	Tài liệu tham khảo	24

1 Giới thiệu

Trong bài thực hành này, em tiến hành cài đặt và cấu hình môi trường Hadoop theo mô hình **Pseudo-Distributed Mode** trên Ubuntu chạy trong WSL2. Sau khi hoàn tất quá trình cài đặt, hệ thống được sử dụng để thực hiện các thao tác trên HDFS và chạy chương trình WordCount bằng Scala sử dụng mô hình MapReduce.

Mục tiêu của bài thực hành bao gồm:

- Cài đặt Java và Hadoop.
- Cấu hình các thành phần của Hadoop.
- Khởi tạo và vận hành cụm Hadoop.
- Thực hiện các thao tác quản lý dữ liệu trên HDFS.
- Chạy chương trình kiểm tra (Verification).
- Xây dựng và chạy chương trình WordCount bằng MapReduce.

2 Cài đặt môi trường

2.1 Cài đặt Java

```
vandiemmy@My:~$ java -version
openjdk version "11.0.31" 2026-04-21
OpenJDK Runtime Environment (build 11.0.31+11-post-1ubuntu1-24.04.2-Ubuntu)
OpenJDK 64-Bit Server VM (build 11.0.31+11-post-1ubuntu1-24.04.2-Ubuntu, mixed mode, sharing)
```

Hình 1: Kiểm tra phiên bản Java

Kết quả trả về cho thấy hệ thống đã nhận diện được Java và hiển thị đúng phiên bản đang sử dụng.

2.2 Cấu hình SSH

```
vandiemmy@My:~$ ssh localhost
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.18.33.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sun Jul  5 09:48:40 UTC 2026

System load:  0.02               Processes:            47
Usage of /:   0.9% of 1006.85GB  Users logged in:     1
Memory usage: 2%                IPv4 address for eth0: 172.26.189.173
Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

1 updates could not be installed automatically. For more details,
see /var/log/unattended-upgrades/unattended-upgrades.log
Last login: Wed Jul  1 14:22:36 2026 from 127.0.0.1
```

Hình 2: Kiểm tra kết nối SSH tới localhost

Cần cấu hình SSH đăng nhập không cần mật khẩu các script như `start-dfs.sh` hoặc `start-yarn.sh` sẽ tự động khởi động các daemon.

2.3 Cài đặt Hadoop

```
vandiemmy@My:~$ hadoop version
Hadoop 3.3.6
Source code repository https://github.com/apache/hadoop.git -r 1be78238728da9266a4f88195058f08fd012bf9c
Compiled by ubuntu on 2023-06-18T08:22Z
Compiled on platform linux-x86_64
Compiled with protoc 3.7.1
From source with checksum 5652179ad55f76cb287d9c633bb53bbd
This command was run using /home/vandiemmy/hadoop/share/hadoop/common/hadoop-common-3.3.6.jar
```

Hình 3: Kiểm tra Hadoop

3 Cấu hình các tập tin Hadoop

3.1 hadoop-env.sh

```
vandiemmy@My:~$ cat $HADOOP_HOME/etc/hadoop/hadoop-env.sh | grep JAVA_HOME
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
# Technically, the only required environment variable is JAVA_HOME.
```

Hình 4: Cấu hình JAVA_HOME

File khai báo các biến môi trường cần thiết cho Hadoop. Có đổi lại JAVA_HOME để daemon sử dụng giá trị tìm chương trình Java.

3.2 core-site.xml

```
vandiemmy@My:~$ cat $HADOOP_HOME/etc/hadoop/core-site.xml
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<!--
  Licensed under the Apache License, Version 2.0 (the "License");
  you may not use this file except in compliance with the License.
  You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

  Unless required by applicable law or agreed to in writing, software
  distributed under the License is distributed on an "AS IS" BASIS,
  WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
  See the License for the specific language governing permissions and
  limitations under the License. See accompanying LICENSE file.
-->

<!-- Put site-specific property overrides in this file. -->

<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
</configuration>
```

Hình 5: Cấu hình core-site.xml

Chứa các cấu hình chung của Hadoop và xác định hệ thống tập tin mặc định. Gán `fs.defaultFS` bằng `hdfs://localhost:9000` quy định địa chỉ của NameNode. Nhờ đó Hadoop sẽ tự động kết nối tới NameNode đang chạy tại cổng 9000 thay vì hệ thống tập tin cục bộ của Linux.

3.3 hdfs-site.xml

```
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>file:///home/vandiemmy/hadoopdata/hdfs/namenode</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>file:///home/vandiemmy/hadoopdata/hdfs/datanode</value>
  </property>
</configuration>
```

Hình 6: Cấu hình hdfs-site.xml

Chứa các cấu hình liên quan đến HDFS, bao gồm số lượng bản sao của dữ liệu và vị trí lưu trữ dữ liệu trên máy.

- **dfs.replication:** Quy định số lượng bản sao (replica) của mỗi block dữ liệu trong HDFS.
- **dfs.namenode.name.dir:** Xác định thư mục lưu trữ metadata của HDFS. NameNode sử dụng thư mục này để quản lý toàn bộ hệ thống HDFS.
- **dfs.datanode.data.dir:** Quy định thư mục lưu trữ dữ liệu thực tế của HDFS. Mỗi khi người dùng upload một file lên HDFS, dữ liệu sẽ được chia thành các block và lưu trong thư mục này.

3.4 mapred-site.xml

```
<configuration>

  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>

  <property>
    <name>yarn.app.mapreduce.am.env</name>
    <value>HADOOP_MAPRED_HOME=/home/vandiemmy/hadoop</value>
  </property>

  <property>
    <name>mapreduce.map.env</name>
    <value>HADOOP_MAPRED_HOME=/home/vandiemmy/hadoop</value>
  </property>

  <property>
    <name>mapreduce.reduce.env</name>
    <value>HADOOP_MAPRED_HOME=/home/vandiemmy/hadoop</value>
  </property>

</configuration>
```

Hình 7: Cấu hình mapred-site.xml

Dùng để cấu hình môi trường thực thi của MapReduce.

Gán `mapreduce.framework.name = yarn` nghĩa là các chương trình MapReduce sẽ được thực thi thông qua YARN thay vì chạy trực tiếp trên Local Mode.

3.5 yarn-site.xml

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>

  <property>
    <name>yarn.resourcemanager.hostname</name>
    <value>localhost</value>
  </property>

  <property>
    <name>yarn.resourcemanager.webapp.address</name>
    <value>0.0.0.0:8088</value>
  </property>
</configuration>
```

Hình 8: Cấu hình yarn-site.xml

Chứa các cấu hình của hệ thống quản lý tài nguyên YARN.

- `yarn.resourcemanager.hostname = localhost` xác định địa chỉ của ResourceManager. NodeManager sẽ sử dụng địa chỉ này để kết nối và nhận nhiệm vụ từ ResourceManager.
- `yarn.nodemanager.aux-services = mapreduce_shuffle`: Thuộc tính này khai báo các dịch vụ phụ trợ của NodeManager. Được sử dụng để hỗ trợ giai đoạn Shuffle của MapReduce.

4 Khởi tạo và khởi động cụm Hadoop

4.1 Format NameNode

```

2026-07-05 10:21:55,799 INFO snapshot.SnapshotManager: SkipList is disabled
2026-07-05 10:21:55,802 INFO util.GSet: Computing capacity for map cachedBlocks
2026-07-05 10:21:55,802 INFO util.GSet: VM type = 64-bit
2026-07-05 10:21:55,802 INFO util.GSet: 0.25% max memory 3.9 GB = 9.9 MB
2026-07-05 10:21:55,802 INFO util.GSet: capacity = 2^20 = 1048576 entries
2026-07-05 10:21:55,810 INFO metrics.TopMetrics: NNTop conf: dfs.namenode.top.window.num.buckets = 10
2026-07-05 10:21:55,810 INFO metrics.TopMetrics: NNTop conf: dfs.namenode.top.num.users = 10
2026-07-05 10:21:55,810 INFO metrics.TopMetrics: NNTop conf: dfs.namenode.top.windows.minutes = 1,5,25
2026-07-05 10:21:55,814 INFO namenode.FSNamesystem: Retry cache on namenode is enabled
2026-07-05 10:21:55,814 INFO namenode.FSNamesystem: Retry cache will use 0.03 of total heap and retry cache entry expiry time is 600000 millis
2026-07-05 10:21:55,815 INFO util.GSet: Computing capacity for map NameNodeRetryCache
2026-07-05 10:21:55,815 INFO util.GSet: VM type = 64-bit
2026-07-05 10:21:55,815 INFO util.GSet: 0.0299999999329447746% max memory 3.9 GB = 1.2 MB
2026-07-05 10:21:55,815 INFO util.GSet: capacity = 2^17 = 131072 entries
Re-format filesystem in Storage Directory root= /home/vandiemmy/hadoopdata/hdfs/namenode; location= null ? (Y or N) y
2026-07-05 10:22:03,104 INFO namenode.FSImage: Allocated new BlockPoolId: BP-2145991992-127.0.1.1-1783246923099
2026-07-05 10:22:03,104 INFO common.Storage: Will remove files: [/home/vandiemmy/hadoopdata/hdfs/namenode/current/fsimage_00000000000000000000, /home/vandiemmy/hadoopdata/hdfs/namenode/current/VERSION, /home/vandiemmy/hadoopdata/hdfs/namenode/current/fsimage_00000000000000000000.md5, /home/vandiemmy/hadoopdata/hdfs/namenode/current/seen_txid]
2026-07-05 10:22:03,134 INFO common.Storage: Storage directory /home/vandiemmy/hadoopdata/hdfs/namenode has been successfully formatted.
2026-07-05 10:22:03,162 INFO namenode.FSImageFormatProtobuf: Saving image file /home/vandiemmy/hadoopdata/hdfs/namenode/current/fsimage.ckpt_00000000000000000000 using no compression
2026-07-05 10:22:03,239 INFO namenode.FSImageFormatProtobuf: Image file /home/vandiemmy/hadoopdata/hdfs/namenode/current/fsimage.ckpt_00000000000000000000 of size 404 bytes saved in 0 seconds .

```

Hình 9: Format NameNode thành công

NameNode cần được khởi tạo bằng cách format. Quá trình này tạo ra cấu trúc thư mục và các metadata ban đầu để quản lý hệ thống tập tin phân tán.

4.2 Khởi động các dịch vụ Hadoop

Sau khi NameNode được format, các dịch vụ của Hadoop cần được khởi động để HDFS và YARN có thể hoạt động.

Các lệnh sử dụng:

```

start-dfs.sh
start-yarn.sh
jps

```

```
vandiemmy@My:~$ start-dfs.sh
start-yarn.sh
jps
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [My]
WARNING: YARN_CONF_DIR has been replaced by HADOOP_CONF_DIR. Using value of YARN_CONF_DIR.
Starting resourcemanager
WARNING: YARN_CONF_DIR has been replaced by HADOOP_CONF_DIR. Using value of YARN_CONF_DIR.
Starting nodemanagers
WARNING: YARN_CONF_DIR has been replaced by HADOOP_CONF_DIR. Using value of YARN_CONF_DIR.
40096 NodeManager
39234 DataNode
40499 Jps
39462 SecondaryNameNode
39719 ResourceManager
39054 NameNode
```

Hình 10: Các tiến trình Hadoop sau khi khởi động

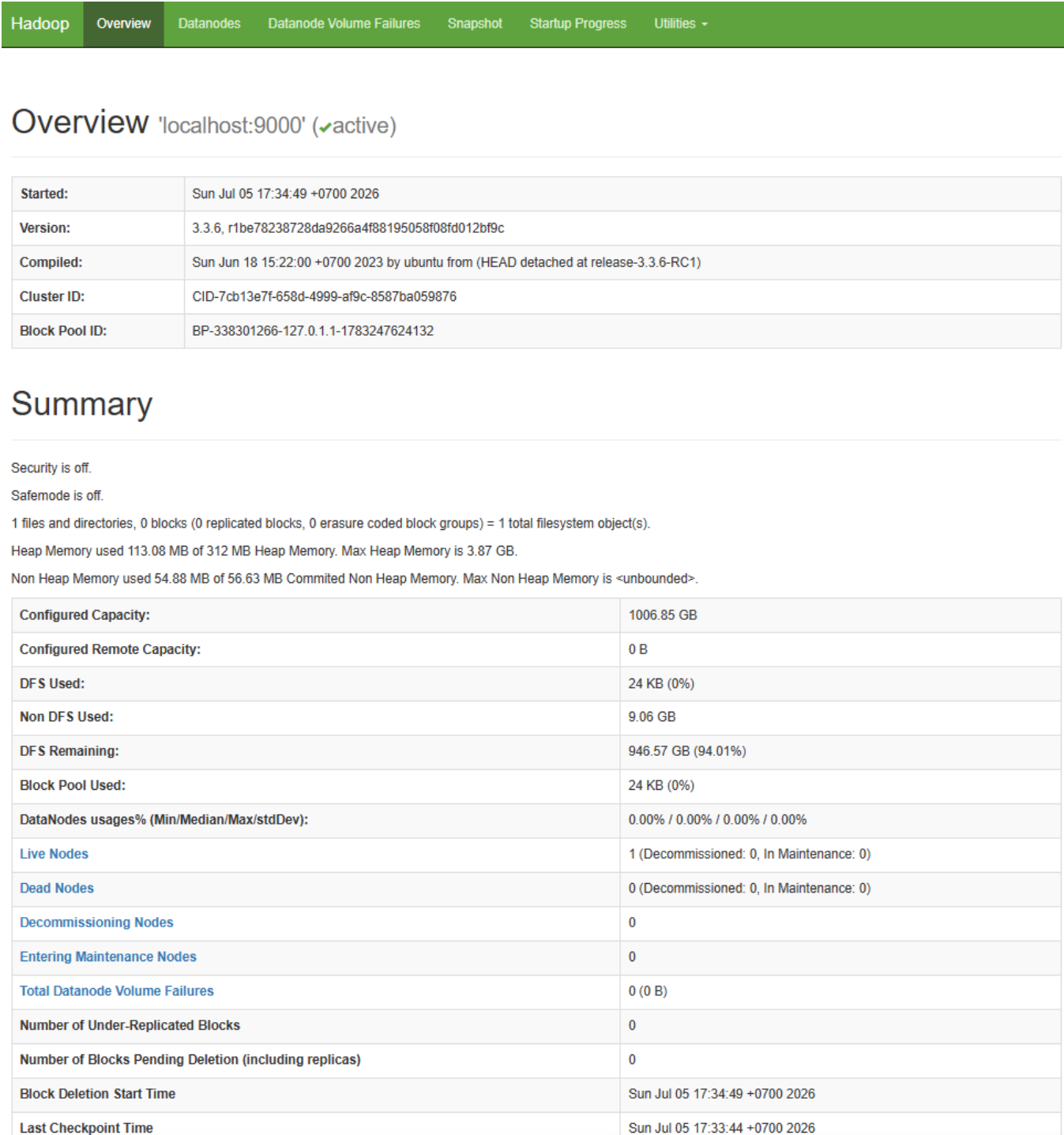
Kết quả của lệnh `jps` hiển thị đầy đủ các tiến trình chính của Hadoop.

4.3 Kiểm tra Web UI

Hadoop cung cấp giao diện Web để theo dõi trạng thái của HDFS và YARN. Hai giao diện được sử dụng trong bài thực hành gồm:

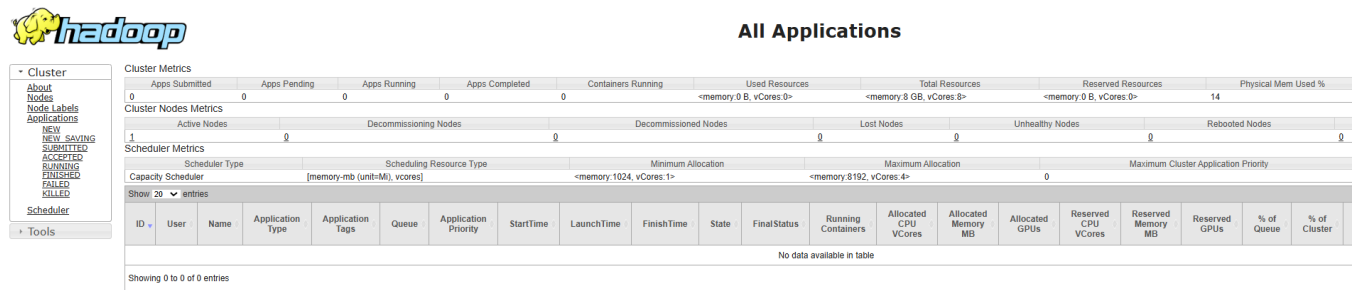
- NameNode Web UI: <http://localhost:9870>
- ResourceManager Web UI: <http://localhost:8088>

NameNode Web UI



Hình 11: NameNode Web UI

ResourceManager Web UI



Hình 12: ResourceManager Web UI

5 Thao tác trên HDFS

5.1 Tạo thư mục trên HDFS và người dùng

```
vandiemmy@My:~$ hdfs dfs -mkdir /hcmus
vandiemmy@My:~$ sudo adduser khtn_23127091
info: Adding user 'khtn_23127091' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group 'khtn_23127091' (1006) ...
info: Adding new user 'khtn_23127091' (1006) with group 'khtn_23127091 (1006)' ...
info: Creating home directory '/home/khtn_23127091' ...
info: Copying files from '/etc/skel' ...
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for khtn_23127091
Enter the new value, or press ENTER for the default
Full Name []:
Room Number []:
Work Phone []:
Home Phone []:
Other []:
Is the information correct? [Y/n] y
info: Adding new user 'khtn_23127091' to supplemental / extra groups 'users' ...
info: Adding user 'khtn_23127091' to group 'users' ...
vandiemmy@My:~$ hdfs dfs -mkdir /hcmus/23127091
```

Hình 13: Tạo thư mục trên HDFS và tạo user

Tạo thư mục /hcmus trên HDFS để lưu trữ dữ liệu và tạo thư mục riêng theo MSSV.
Dùng `sudo adduser khtn_23127091` để cần tạo người dùng.

5.2 Tải dữ liệu lên HDFS

```
vandiemmy@My:~$ echo "helloooo" > file.txt
vandiemmy@My:~$ hdfs dfs -put file.txt /hcmus/23127091/
vandiemmy@My:~$ hdfs dfs -chmod 744 /hcmus/23127091
vandiemmy@My:~$ hdfs dfs -chown khtn_23127091 /hcmus/23127091
vandiemmy@My:~$ hdfs dfs -ls -R /hcmus
drwxr--r-- - khtn_23127091 supergroup          0 2026-07-05 10:50 /hcmus/23127091
-rw-r--r-- 1 vandiemmy supergroup          9 2026-07-05 10:50 /hcmus/23127091/file.txt
```

Hình 14: Upload dữ liệu lên HDFS

Sau khi tạo thư mục, cần tải dữ liệu từ máy cục bộ lên HDFS. Lệnh `-put` sao chép dữ liệu từ hệ thống tập tin của Linux vào HDFS.

Thư mục cần được cấp quyền 744. Sau khi tạo người dùng mới, thư mục trên HDFS cần được chuyển quyền sở hữu. Lệnh `-chown` thay đổi chủ sở hữu của thư mục trên HDFS.

Sau khi thực hiện, tất cả các thao tác quản lý đối với thư mục sẽ thuộc quyền của người dùng:

5.3 Kiểm tra kết quả

Sau khi hoàn thành các thao tác trên, sử dụng lệnh `hdfs dfs -ls -R /hcmus` để kiểm tra toàn bộ cây thư mục.

```
vandiemmy@My:~$ hdfs dfs -ls -R /hcmus
drwxr--r--  - khtn_23127091 supergroup      0 2026-07-05 10:50 /hcmus/23127091
-rw-r--r--  1 vandiemmy    supergroup      9 2026-07-05 10:50 /hcmus/23127091/file.txt
```

Hình 15: Kiểm tra cấu trúc thư mục trên HDFS

6 Chạy chương trình Verification

6.1 Lệnh thực hiện

Lệnh chạy chương trình:

```
java -jar hadoop-test.jar 9000 /hcmus/23127091
```

Trong đó:

- **9000**

Là cổng (port) của NameNode đã được cấu hình trong thuộc tính

```
fs.defaultFS = hdfs://localhost:9000.
```

Thông qua cổng này, chương trình sẽ kết nối tới HDFS.

- **/hcmus/23127091**

Là đường dẫn tới thư mục của sinh viên trên HDFS.

Chương trình sẽ kiểm tra xem thư mục này có tồn tại hay không, đồng thời xác minh quyền truy cập và chủ sở hữu của thư mục.

6.2 Kết quả thực hiện

```
vandiemmy@My:/mnt/d/HK3_3/Big Data/Lab1$ HADOOP_USER_NAME=khtn_23127091 java -jar hadoop-test.jar 9000 /hcmus/23127091
Trying to read /hcmus/23127091
log4j:WARN No appenders could be found for logger (org.apache.hadoop.util.Shell).
log4j:WARN Please initialize the log4j system properly.
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more info.
Found hdfs://localhost:9000/hcmus/23127091/file.txt
Your student ID: 23127091 (ensure it matches your student ID)
The first method to get MAC address is failed: Could not get network interface
Trying the alternative method
The first method to get MAC address is failed: Could not get network interface
Trying the alternative method
File written at /mnt/d/HK3_3/Big Data/Lab1/23127091_verification.txt
```

Hình 16: Kết quả chạy Verification JAR

Sau khi thực hiện lệnh, chương trình kết nối thành công tới HDFS và tiến hành kiểm tra các yêu cầu của bài thực hành.

Nếu toàn bộ điều kiện đều chính xác, chương trình sẽ tạo file xác nhận trên máy cục bộ.

6.3 Kết quả

```
vandiemmy@My:/mnt/d/HK3_3/Big Data/Lab1$ cat 23127091_verification.txt
MAC=00-15-5D-FD-8E-4D
ac5f0f6d994c68673a4297c6ca732172b51c8619717624b5e9c72c493d356971vandiemmy@My:/mnt/d/HK3_3/Big Data/Lab1$ |
```

Hình 17: Nội dung file verification

7 Chương trình WordCount sử dụng MapReduce

7.1 Yêu cầu bài toán

Đề bài yêu cầu đọc một tập tin văn bản từ HDFS và thống kê số lượng từ có ký tự đầu tiên thuộc tập:

$$\{f, i, t, h, c, m, u, s\}$$

Đối với mỗi từ trong tập dữ liệu:

- Tách từ theo khoảng trắng.
- Lấy ký tự đầu tiên của từ.
- Chuyển ký tự về chữ thường (nếu cần).
- Nếu ký tự thuộc tập yêu cầu thì tăng bộ đếm tương ứng.

Kết quả cuối cùng được ghi ra HDFS dưới dạng các cặp:

(letter, count)

Sau đó dữ liệu được xuất sang file `results.txt` theo đúng thứ tự:

f, i, t, h, c, m, u, s.

7.2 Thiết kế chương trình

Chương trình được xây dựng bằng ngôn ngữ Scala theo mô hình MapReduce của Hadoop. Project gồm hai thành phần chính là tập tin cấu hình `build.sbt` và mã nguồn `WordCount.scala`. Trong đó, `build.sbt` chịu trách nhiệm cấu hình môi trường biên dịch và đóng gói project, còn `WordCount.scala` hiện thực toàn bộ thuật toán MapReduce để xử lý dữ liệu.

7.2.1 Tập tin `build.sbt`

Project được quản lý bằng công cụ `sbt` (Scala Build Tool). Tập tin `build.sbt` khai báo các thông tin cần thiết để biên dịch chương trình và tạo file JAR có thể thực thi trên Hadoop.

Trong bài thực hành, project sử dụng Scala phiên bản 2.12.18, tương thích với Hadoop phiên bản 3.3.6. Thư viện `hadoop-client` được thêm vào để cung cấp các lớp cần thiết cho việc xây dựng chương trình MapReduce như `Mapper`, `Reducer`, `Job` và các kiểu dữ liệu của Hadoop.

Ngoài ra, project sử dụng plugin `sbt-assembly` để đóng gói toàn bộ mã nguồn và các thư viện phụ thuộc thành một file JAR duy nhất. Điều này giúp chương trình có thể được triển khai trực tiếp trên Hadoop mà không cần cài đặt thêm các thư viện bên ngoài.

Trong quá trình build, một số tệp như `META-INF` và `module-info.class` được loại bỏ thông qua `MergeStrategy` nhằm tránh xung đột giữa các thư viện khi đóng gói project.

```

ThisBuild / scalaVersion := "2.12.18"

lazy val root = (project in file("."))
  .settings(
    name := "WordCount",
    version := "1.0",

    libraryDependencies ++= Seq(
      "org.apache.hadoop" % "hadoop-client" % "3.3.6"
    ),

    assembly / assemblyMergeStrategy := {
      case PathList("META-INF", _) => MergeStrategy.discard
      case "module-info.class"      => MergeStrategy.discard
      case _                        => MergeStrategy.first
    }
  )
  .enablePlugins(AssemblyPlugin)

```

Hình 18: Nội dung tập tin build.sbt

7.2.2 Lớp FilteredMapper

Lớp `FilteredMapper` kế thừa lớp `Mapper` của Hadoop và chịu trách nhiệm xử lý dữ liệu đầu vào. Mỗi dòng văn bản được đọc từ HDFS sẽ được tách thành các từ dựa trên khoảng trắng. Đối với mỗi từ, chương trình lấy ký tự đầu tiên, chuyển về chữ thường và kiểm tra xem ký tự đó có thuộc tập

$$\{f, i, t, h, c, m, u, s\}$$

hay không. Nếu không thuộc tập ký tự yêu cầu thì từ đó sẽ bị bỏ qua.

Đối với các từ hợp lệ, Mapper không phát sinh trực tiếp khóa là ký tự đầu tiên mà thêm một chỉ số thứ tự vào trước ký tự. Chỉ số này được xác định theo chuỗi

"fithcmus"

Ví dụ:

$$food \rightarrow (0f, 1)$$

$$human \rightarrow (3h, 1)$$

Việc bổ sung chỉ số thứ tự giúp Hadoop tự động sắp xếp khóa theo đúng thứ tự yêu cầu trong giai đoạn Shuffle và Sort. Nhờ đó, kết quả cuối cùng sẽ được xuất theo đúng thứ tự mà không cần xử lý bổ sung sau khi chương trình kết thúc.

```
class FilteredMapper extends Mapper[LongWritable, Text, Text, IntWritable] {
  val targetLetters = Set('f', 'i', 't', 'h', 'c', 'm', 'u', 's')
  val letterOrder: Map[Char, Int] =
    targetLetters.toSeq.sortBy(c => "fithcmus".indexOf(c)).zipWithIndex.toMap
  val one = new IntWritable(1)
  val keyOut = new Text()

  override def map(key: LongWritable, value: Text,
    context: Mapper[LongWritable, Text, Text, IntWritable]#Context): Unit = {
    val words = value.toString.split("\\s+")
    for (w <- words if w.nonEmpty) {
      val firstChar = w.charAt(0).toLowerCase
      if (targetLetters.contains(firstChar)) {
        val order = letterOrder(firstChar)
        keyOut.set(s"$order$firstChar")
        context.write(keyOut, one)
      }
    }
  }
}
```

Hình 19: Cài đặt lớp FilteredMapper

7.2.3 Lớp IntSumCombiner

Lớp IntSumCombiner được sử dụng như một Combiner để giảm lượng dữ liệu trung gian truyền từ Mapper sang Reducer.

Đối với mỗi khóa, Combiner cộng tạm thời các giá trị ngay trên máy thực thi Mapper.

Ví dụ:

$$(0f, 1), (0f, 1), (0f, 1)$$

sẽ được gộp thành

$$(0f, 3)$$

Việc sử dụng Combiner không làm thay đổi kết quả cuối cùng nhưng giúp giảm số lượng dữ liệu cần truyền qua mạng, từ đó cải thiện hiệu năng của chương trình.

```
class IntSumCombiner extends Reducer[Text, IntWritable, Text, IntWritable] {
  override def reduce(key: Text, values: java.lang.Iterable[IntWritable],
    context: Reducer[Text, IntWritable, Text, IntWritable]#Context): Unit = {
    val sum = values.asScala.map(_.get()).sum
    context.write(key, new IntWritable(sum))
  }
}
```

Hình 20: Cài đặt lớp IntSumCombiner

7.2.4 Lớp IntSumReducer

Sau khi hoàn thành giai đoạn Shuffle và Sort, Reducer nhận các khóa đã được nhóm và sắp xếp từ Hadoop.

Đối với mỗi khóa, Reducer cộng tất cả các giá trị để thu được tổng số lần xuất hiện của ký tự tương ứng.

Ví dụ:

$$(of, 2), (of, 5)$$

sẽ được xử lý thành

$$(of, 7)$$

Trước khi ghi kết quả ra HDFS, chương trình loại bỏ ký tự chỉ thứ tự ở đầu khóa bằng phương thức

$$\text{substring}(1)$$

Do đó, kết quả cuối cùng chỉ còn lại ký tự và số lần xuất hiện, ví dụ

$$(f, 7)$$

Trong khi đó, thứ tự các dòng vẫn được giữ nguyên theo đúng yêu cầu của đề bài nhờ cơ chế sắp xếp khóa của Hadoop.

```
class IntSumReducer extends Reducer[Text, IntWritable, Text, IntWritable] {  
  val keyOut = new Text()  
  
  override def reduce(key: Text, values: java.lang.Iterable[IntWritable],  
    context: Reducer[Text, IntWritable, Text, IntWritable]#Context): Unit = {  
    val sum = values.asScala.map(_.get()).sum  
    keyOut.set(key.toString.substring(1))  
    context.write(keyOut, new IntWritable(sum))  
  }  
}
```

Hình 21: Cài đặt lớp IntSumReducer

7.2.5 WordCount

WordCount chứa hàm main và là điểm bắt đầu của chương trình MapReduce.

Trong hàm main, chương trình khởi tạo đối tượng Configuration và tạo một Job mới để thực thi trên Hadoop. Tiếp theo, chương trình lần lượt khai báo các lớp FilteredMapper, IntSumCombiner và IntSumReducer, đồng thời xác định kiểu dữ liệu đầu ra của Job là Text và IntWritable.

Cuối cùng, chương trình cấu hình đường dẫn dữ liệu đầu vào và đầu ra trên HDFS thông qua các lớp FileInputFormat và FileOutputFormat. Sau khi hoàn tất cấu hình, phương thức

```
job.waitForCompletion(true)
```

được gọi để gửi Job tới Hadoop và chờ quá trình xử lý kết thúc. Nếu Job thực thi thành công, chương trình sẽ trả về mã thoát 0; ngược lại sẽ trả về mã 1 để thông báo có lỗi xảy ra.

7.3 Thực thi chương trình

Sau khi build thành công, chương trình được chạy bằng lệnh:

```
hadoop jar target/scala-2.12/WordCount-assembly-1.0.jar \  
/hcmus/23127091/words.txt \  
/hcmus/23127091/output
```

```
object WordCount {  
  def main(args: Array[String]): Unit = {  
    val conf = new Configuration()  
    val job = Job.getInstance(conf, "WordCount")  
    job.setJarByClass(this.getClass)  
  
    job.setMapperClass(classOf[FilteredMapper])  
    job.setCombinerClass(classOf[IntSumCombiner])  
    job.setReducerClass(classOf[IntSumReducer])  
  
    job.setOutputKeyClass(classOf[Text])  
    job.setOutputValueClass(classOf[IntWritable])  
  
    FileInputFormat.addInputPath(job, new Path(args(0)))  
    FileOutputFormat.setOutputPath(job, new Path(args(1)))  
  
    System.exit(if (job.waitForCompletion(true)) 0 else 1)  
  }  
}
```

Hình 22: Hàm main của chương trình

Kết quả

```
vandiemmy@My:/mnt/d/HW3_3/Big Data/Lab1$ hadoop jar target/scala-2.12/WordCount-assembly-1.0.jar WordCount /hcmus/23127091/words.txt /hcmus/23127091/output
2026-07-05 14:41:02,190 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at localhost/127.0.0.1:8032
2026-07-05 14:41:02,488 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your
application with ToolRunner to remedy this.
2026-07-05 14:41:02,496 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/vandiemmy/.staging/job_1783247698963
_0001
2026-07-05 14:41:04,644 INFO input.FileInputFormat: Total input files to process : 1
2026-07-05 14:41:04,726 INFO mapreduce.JobSubmitter: number of splits:1
2026-07-05 14:41:04,855 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1783247698963_0001
2026-07-05 14:41:04,855 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-07-05 14:41:04,960 INFO conf.Configuration: resource-types.xml not found
2026-07-05 14:41:04,960 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-07-05 14:41:04,662 INFO impl.YarnClientImpl: Submitted application application_1783247698963_0001
2026-07-05 14:41:04,685 INFO mapreduce.Job: The url to track the job: http://My.localdomain:8088/proxy/application_1783247698963_0001/
2026-07-05 14:41:04,686 INFO mapreduce.Job: Running job: job_1783247698963_0001
2026-07-05 14:41:11,759 INFO mapreduce.Job: Job job_1783247698963_0001 running in uber mode : false
2026-07-05 14:41:11,759 INFO mapreduce.Job: map 0% reduce 0%
2026-07-05 14:41:15,793 INFO mapreduce.Job: map 100% reduce 0%
2026-07-05 14:41:20,819 INFO mapreduce.Job: map 100% reduce 100%
2026-07-05 14:41:20,824 INFO mapreduce.Job: Job job_1783247698963_0001 completed successfully
```

Hình 23: Log thực thi MapReduce

Giải thích

Trong quá trình thực thi, Hadoop tự động:

- Khởi tạo Job.
- Phân chia dữ liệu đầu vào.
- Thực hiện các Mapper.
- Thực hiện Shuffle và Sort.
- Chạy Reducer.
- Ghi kết quả ra HDFS.

Khi log hiển thị trạng thái:

```
map 100%
reduce 100%
completed successfully
```

điều đó cho thấy chương trình đã chạy thành công.

7.4 Kết quả trên HDFS

Sau khi chương trình kết thúc, kiểm tra kết quả bằng lệnh:

```
hdfs dfs -cat /hcmus/23127091/output/part-r-00000
```

Kết quả

```
Bytes Written=64
vandiemmy@My:/mnt/d/HK3_3/Big Data/Lab1$ hdfs dfs -cat /hcmus/23127091/output/part-r-00000
c      38934
f      15870
h      18662
i      15220
m      25194
s      50571
t      25223
u      23790
```

Hình 24: Kết quả WordCount trên HDFS

Giải thích

File

part-r-00000

là kết quả do Reducer tạo ra.

Mỗi dòng của file bao gồm:

ký tự số lần xuất hiện

đây chính là kết quả thống kê cuối cùng của chương trình.

8 Lỗi gặp phải và cách khắc phục

8.1 Lỗi JAVA_HOME không tồn tại

Mô tả lỗi

Sau khi cấu hình Hadoop và thực hiện lệnh `start-dfs.sh` hệ thống hiển thị thông báo lỗi mặc dù đã chỉnh JAVA_HOME lúc sửa file `hadoop-env.sh`:

```
vandiemmy@My:/mnt/d/HK3_3/Big Data/Lab1$ start-dfs.sh
ERROR: JAVA_HOME /usr/java/testing does not exist.
```

Hình 25: Lỗi JAVA_HOME không tồn tại

Nguyên nhân

Qua kiểm tra lại tập tin `hadoop-env.sh`, em nhận thấy biến `JAVA_HOME` đang trỏ tới một đường dẫn khác.

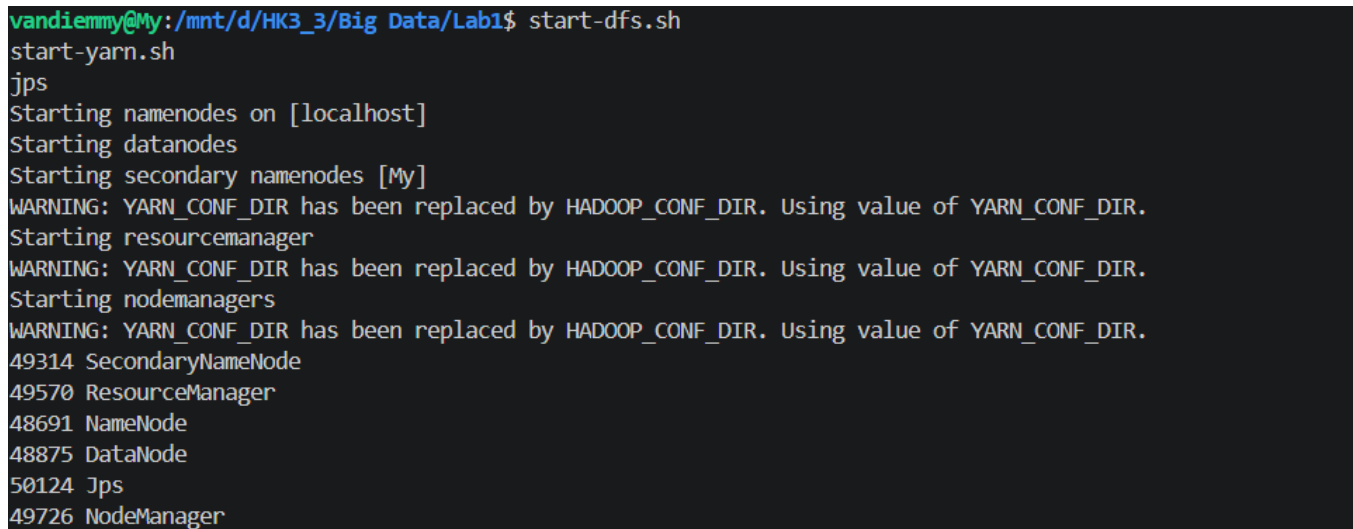
Trong khi đó, Java đã được cài đặt tại một vị trí khác nên Hadoop không thể tìm thấy chương trình Java để khởi động các daemon.

Cách khắc phục

Đầu tiên, em xác định đường dẫn thực tế của Java sau đó chỉnh sửa tập tin `hadoop-env.sh` và cập nhật biến `JAVA_HOME` thành đúng đường dẫn chứa Java.

Kết quả

Sau khi cập nhật đúng đường dẫn Java, Hadoop nhận diện được Java và không còn xuất hiện thông báo lỗi.



```
vandiemmy@My:/mnt/d/HK3_3/Big Data/Lab1$ start-dfs.sh
start-yarn.sh
jps
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [My]
WARNING: YARN_CONF_DIR has been replaced by HADOOP_CONF_DIR. Using value of YARN_CONF_DIR.
Starting resourcemanager
WARNING: YARN_CONF_DIR has been replaced by HADOOP_CONF_DIR. Using value of YARN_CONF_DIR.
Starting nodemanagers
WARNING: YARN_CONF_DIR has been replaced by HADOOP_CONF_DIR. Using value of YARN_CONF_DIR.
49314 SecondaryNameNode
49570 ResourceManager
48691 NameNode
48875 DataNode
50124 Jps
49726 NodeManager
```

Hình 26: Hadoop hoạt động bình thường sau khi sửa `JAVA_HOME`

8.2 Lỗi Incompatible ClusterID

Mô tả lỗi

Sau khi thực hiện `start-dfs.sh` NameNode khởi động bình thường nhưng DataNode không thể chạy.

```
vandiemmy@My:/mnt/d/HK3_3/Big Data/Lab1$ start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [My]
vandiemmy@My:/mnt/d/HK3_3/Big Data/Lab1$ start-yarn.sh
WARNING: YARN_CONF_DIR has been replaced by HADOOP_CONF_DIR. Using value of YARN_CONF_DIR.
Starting resourcemanager
WARNING: YARN_CONF_DIR has been replaced by HADOOP_CONF_DIR. Using value of YARN_CONF_DIR.
Starting nodemanagers
WARNING: YARN_CONF_DIR has been replaced by HADOOP_CONF_DIR. Using value of YARN_CONF_DIR.
vandiemmy@My:/mnt/d/HK3_3/Big Data/Lab1$ jps
113850 NameNode
115117 Jps
114254 SecondaryNameNode
```

Hình 27: Thiếu DataNode

Nguyên nhân

NameNode và DataNode đều lưu thông tin về ClusterID trong thư mục dữ liệu của mình.

Trong quá trình cài đặt, em đã format NameNode nhiều lần nhưng chưa xóa dữ liệu cũ của DataNode.

Do đó:

- NameNode tạo ClusterID mới.
- DataNode vẫn lưu ClusterID cũ.
- Hai giá trị không trùng nhau nên DataNode từ chối kết nối.


```

2026-07-09 15:58:10,869 INFO org.apache.hadoop.metrics2.impl.MetricsConfig: Loaded properties from hadoop-metrics2.properties
2026-07-09 15:58:10,970 INFO org.apache.hadoop.metrics2.impl.MetricsSystemImpl: Scheduled Metric snapshot period at 10 second(s).
2026-07-09 15:58:10,970 INFO org.apache.hadoop.metrics2.impl.MetricsSystemImpl: DataNode metrics system started
2026-07-09 15:58:11,107 INFO org.apache.hadoop.hdfs.server.common.Util: dfs.datanode.fileio.profiling.sampling.percentage set to 0. Disabling
file IO profiling
--
java.io.IOException: Incompatible clusterIDs in /home/vandiemmy/hadoopdata/hdfs/datanode: namenode clusterID = CID-3086f9a6-1e35-4766-9542-214
c1353547d; datanode clusterID = CID-7cb13e7f-658d-4999-af9c-8587ba059876
    at org.apache.hadoop.hdfs.server.datanode.DataStorage.doTransition(DataStorage.java:746)
    at org.apache.hadoop.hdfs.server.datanode.DataStorage.loadStorageDirectory(DataStorage.java:296)
    at org.apache.hadoop.hdfs.server.datanode.DataStorage.loadDataStorage(DataStorage.java:409)
    at org.apache.hadoop.hdfs.server.datanode.DataStorage.addStorageLocations(DataStorage.java:389)
    at org.apache.hadoop.hdfs.server.datanode.DataStorage.recoverTransitionRead(DataStorage.java:561)
    at org.apache.hadoop.hdfs.server.datanode.DataNode.initStorage(DataNode.java:2059)
    at org.apache.hadoop.hdfs.server.datanode.DataNode.initBlockPool(DataNode.java:1995)
    at org.apache.hadoop.hdfs.server.datanode.BPOfferService.verifyAndSetNamespaceInfo(BPOfferService.java:394)
    at org.apache.hadoop.hdfs.server.datanode.BPServiceActor.connectToNNAndHandshake(BPServiceActor.java:312)
    at org.apache.hadoop.hdfs.server.datanode.BPServiceActor.run(BPServiceActor.java:891)
    at java.base/java.lang.Thread.run(Thread.java:1583)
2026-07-09 15:58:11,814 ERROR org.apache.hadoop.hdfs.server.datanode.DataNode: Initialization failed for Block pool <registering> (Datanode Uu
id 4902cdeb-b1ab-47d2-92c7-5c7ed208924b) service to localhost/127.0.0.1:9000. Exiting.
java.io.IOException: All specified directories have failed to load.
    at org.apache.hadoop.hdfs.server.datanode.DataStorage.recoverTransitionRead(DataStorage.java:562)
    at org.apache.hadoop.hdfs.server.datanode.DataNode.initStorage(DataNode.java:2059)
    at org.apache.hadoop.hdfs.server.datanode.DataNode.initBlockPool(DataNode.java:1995)
    at org.apache.hadoop.hdfs.server.datanode.BPOfferService.verifyAndSetNamespaceInfo(BPOfferService.java:394)
    at org.apache.hadoop.hdfs.server.datanode.BPServiceActor.connectToNNAndHandshake(BPServiceActor.java:312)
    at org.apache.hadoop.hdfs.server.datanode.BPServiceActor.run(BPServiceActor.java:891)
    at java.base/java.lang.Thread.run(Thread.java:1583)
2026-07-09 15:58:11,815 WARN org.apache.hadoop.hdfs.server.datanode.DataNode: Ending block pool service for: Block pool <registering> (Datanod
e Uuid 4902cdeb-b1ab-47d2-92c7-5c7ed208924b) service to localhost/127.0.0.1:9000
2026-07-09 15:58:11,815 INFO org.apache.hadoop.hdfs.server.datanode.DataNode: Removed Block pool <registering> (Datanode Uuid 4902cdeb-b1ab-47
d2-92c7-5c7ed208924b)
2026-07-09 15:58:13,816 WARN org.apache.hadoop.hdfs.server.datanode.DataNode: Exiting Datanode
2026-07-09 15:58:13,818 INFO org.apache.hadoop.hdfs.server.datanode.DataNode: SHUTDOWN_MSG:
/*****
SHUTDOWN_MSG: Shutting down DataNode at My/127.0.1.1
*****/

```

Hình 28: Lỗi Incompatible ClusterID

Cách khắc phục

Em dừng toàn bộ Hadoop bằng các lệnh sau đó xóa toàn bộ dữ liệu HDFS cũ và format lại NameNode sau đó khởi động lại Hadoop.

```

vandiemmy@My:/mnt/d/HK3_3/Big Data/Lab1$ stop-dfs.sh
stop-yarn.sh
Stopping namenodes on [localhost]
Stopping datanodes
Stopping secondary namenodes [My]
WARNING: YARN_CONF_DIR has been replaced by HADOOP_CONF_DIR. Using value of YARN_CONF_DIR.
Stopping nodemanagers
WARNING: YARN_CONF_DIR has been replaced by HADOOP_CONF_DIR. Using value of YARN_CONF_DIR.
Stopping resourcemanager
WARNING: YARN_CONF_DIR has been replaced by HADOOP_CONF_DIR. Using value of YARN_CONF_DIR.
vandiemmy@My:/mnt/d/HK3_3/Big Data/Lab1$ rm -rf ~/hadoopdata/hdfs/namenode/*
rm -rf ~/hadoopdata/hdfs/datanode/*

```

Hình 29: Sửa clusterID

Kết quả

Sau khi xóa dữ liệu cũ và format lại HDFS, DataNode kết nối thành công với NameNode. Lệnh jps hiển thị đầy đủ các tiến trình Hadoop.

```
vandiemmy@My:/mnt/d/HK3_3/Big Data/Lab1$ start-dfs.sh
start-yarn.sh
jps
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [My]
WARNING: YARN_CONF_DIR has been replaced by HADOOP_CONF_DIR. Using value of YARN_CONF_DIR.
Starting resourcemanager
WARNING: YARN_CONF_DIR has been replaced by HADOOP_CONF_DIR. Using value of YARN_CONF_DIR.
Starting nodemanagers
WARNING: YARN_CONF_DIR has been replaced by HADOOP_CONF_DIR. Using value of YARN_CONF_DIR.
49314 SecondaryNameNode
49570 ResourceManager
48691 NameNode
48875 DataNode
50124 Jps
49726 NodeManager
```

Hình 30: DataNode hoạt động sau khi sửa lỗi ClusterID

9 Tài liệu tham khảo

1. Apache Hadoop Documentation.

<https://hadoop.apache.org/docs/>

2. Apache Hadoop MapReduce Tutorial.

<https://hadoop.apache.org/docs/stable/hadoop-mapreduce-client/hadoop-mapreduce-client-MapReduceTutorial.html>

3. Apache Hadoop HDFS User Guide.

<https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsUserGuide.html>